

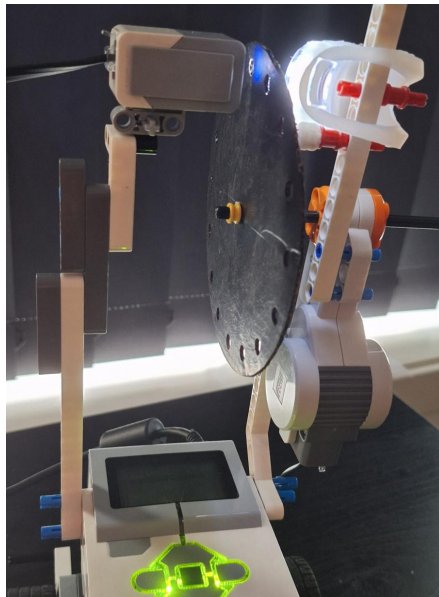


Universitetet
i Stavanger

Elektroingeniør Portefølje

Hongmei Li

19. august 2026



Scan for å se prosjektporteføljen på GitHub

Innhold

Sammendrag	6
Prosjekter	1
1 Optisk Enkoder	1
1.1 Teori	1
1.2 Anvendelse	2
1.3 Oppsett	2
1.4 Metode	3
1.5 Resultat	5
1.6 Kryssverifisering:	7
1.7 Diskusjon og konklusjon:	8
2 Vannlekkasjedetektor	9
2.1 Problemstilling	9
2.2 Teori	9
2.3 Eksperiment	11
2.4 Konklusjon	13
3 Strømkalkulator	14
3.1 Problemstilling	14
3.2 Utviklingsprosess	14
3.3 Funksjon	15

4	Analyse av strømforbruk i Excel og Access	16
4.1	Formål	16
4.2	Databehandling i Access	18
4.3	Analyse og visualisering i Excel	20
4.4	Konklusjon	23
	Refleksjon	24
	Referanser	25
	Vedlegg	26
A	Rushtid	26
B	Spørringsresultat for forbuk per prisområde og forbruksgruppe	27
C	Python og MATLAB koder	27

Figurer

1	Illustrasjon av optisk enkoder[2]	1
2	Enkoder i vannkraftverk	2
3	Egenkonstruert optisk enkoder	3
4	Trapez vs rektangel signal	5
5	Simulering av signal, vinkelhastighet, akselerasjon og posisjon . .	6
6	Rotasjonshastighet i display	7
7	Hvordan ioner i vann leder elektrisk strøm. Blå bobler representerer vannmolyker (KI-generert med brukerjusteringer)	9
8	Kretsen for røykvarsler. Fotodioden (PD) fungerer som lyssensor i alarmen	10
9	Kretsen for vanndetektor. Vann fungerer som en bryter som lukker kretsen ved kontakt	10
10	Lodding av ekstern ledninger	11
11	Kortslutning av røyksensor	12
12	Kortslutning av røyksensor	12
13	Brukergrensesnitt for strømkalkulatoren	15
14	Utdrag av strømdatatabellen	16
15	Geografisk fordeling av prisområdene[5]	16
16	Flytskjema som viser arbeidsprosessen	17
17	Arbeidsprosess i Access	18
18	Relasjon mellom tabellene	19
19	Totalt strømforbruk fordelt på prisområder	20

20	Totalt strømforbruk fordelt på forbruksgrupper	21
21	Strømforbruk fordelt på time i døgnet	22
22	Strømforbruk fordelt på forbruksgrupper og prisområder	22
23	Strømforbruk fordelt på tidspunkt	26
24	Forbruk per prisområde og gruppe	27

Kodeutdrag

1	Kantedeteksjon	4
2	Beregning av integrasjon og bakoverderivasjon	4
3	Filtere hastighet og akselerasjon	5
4	Enkoder	27
5	Elektrisitet kakulator	31

Tabeller

1	Prisområder	18
2	Forbruksgrupper	18
3	Totalt strømforbruk per prisområdet	19
4	Totalt strømforbruk per forbruksgroup	20

Sammendrag

Jeg er en elektroingeniørstudent med interesse for energisystemer, sensorteknologi og dataanalyse. Denne porteføljen presenterer følgende prosjekter som viser både praktisk arbeid og analytiske ferdigheter:

- Optisk enkoder
- Vannlekkasjedetektor
- Strømkalkulator
- Analyse av strømforbruk i Excel og Access

Den optiske enkoderen er videreutviklet fra en tidligere skolerapport. Prosjektet ble satt inn i et større perspektiv for å analysere enkoderens rolle i den helhetlige kraftproduksjonen. Prosjektet ga erfaring med sensorsystemer, signalbehandling og numerisk analyse, samt praktisk bygging og testing av målesystemer.

Vannlekkasjedetektoren er et selvstendig prosjekt inspirert av en reell problemstilling i hjemmet. Løsningen ble utviklet ved å modifisere en eksisterende røykvarsler. Kretsanalyse av røykvarsleren var sentral for å forstå hvordan små endringer i kretsen kan endre produktets bruksområde.

Strømkalkulatoren er en skrivebordsapplikasjon utviklet i Python for å beregne strømforbruk og kostnader på en enkel og brukervennlig måte.

Analyse av strømforbruk er et egeninitiert prosjekt som bygger på databasekunnskap fra min tidligere logistikkutdanning. Prosjektet viser en tverrfaglig tilnærming til problemløsning gjennom kombinasjon av databaser, dataanalyse og visualisering.

Prosjektene reflekterer ferdigheter som er relevante for arbeid innen elkraft, sensorteknologi og systemutvikling.

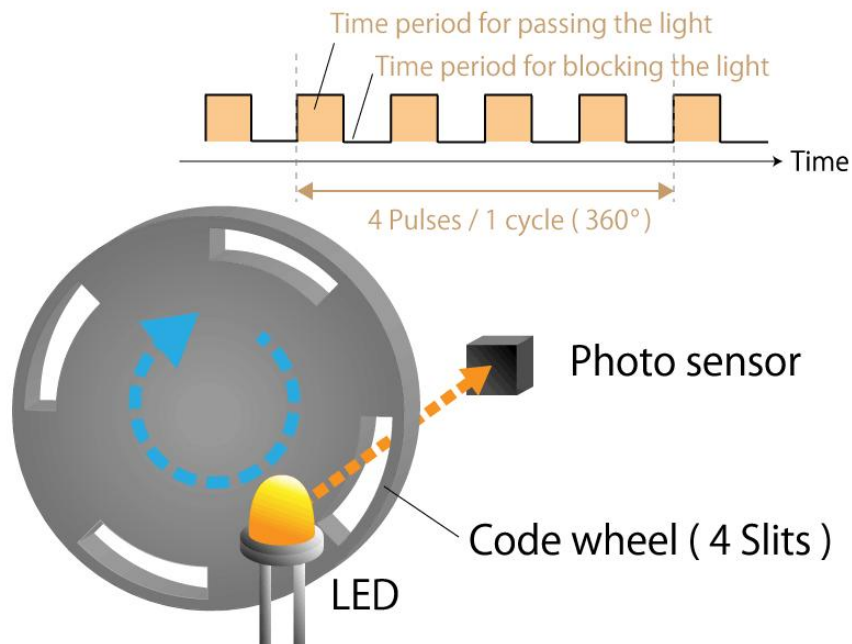
Prosjekter

1 Optisk Enkoder

Prosjektet utvikler et system for å måle rotasjonshastighet. Målet er å få ferdigheter i signalbehandling og numerisk analyse i MATLAB.

1.1 Teori

Optisk enkoder er en elektromekanisk enhet som benytter en lyskilde, fotodetektorer og en skive med hull langs kanten for å konvertere mekanisk bevegelse til elektriske signaler[1]. Hullene på skiven er jevnt fordelt med vinkel θ . Når skiven roterer, blir lyset som passerer gjennom hullene periodisk avbrutt. Dette blir registrert av sensor som et tidsvarierende signal, slik vist i figur 1. Når lys passerer gjennom et hull og treffer sensoren, får signalet verdi 1, ellers verdi 0. Det ideelle enkodersignalet er et firkantsignal, og ved å analysere dette signalet kan vinkelposisjonen S , rotasjonshastighet ω og vinkelakselerasjonen α beregnes.

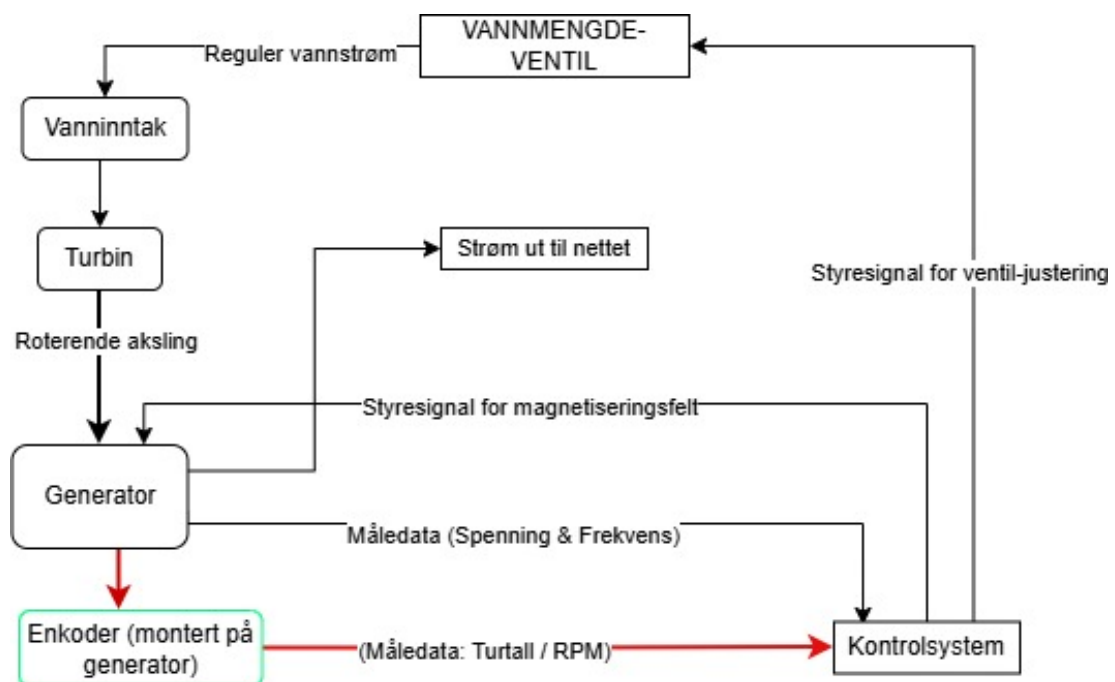


Figur 1: Illustrasjon av optisk enkoder[2]

1.2 Anvendelse

I elkraft og automasjon brukes optiske enkodere til å måle rotasjonen til motor- og generatoraksler. Enkoderen er montert direkte på akselen. Når akslingen roterer, sender sensoren pulser tilbake til et kontrollsystem, som kan beregne to kritiske faktorer: posisjon og hastighet. Denne sanntidsdata brukes blant annet i PID-regulering av motorer, der hastighet og posisjon kontinuerlig justeres for å oppnå ønsket ytelse. I vann- og vindkraft overvåker enkoderen rotasjons hastigheten til turbinen. Kraftverkstyringen bruker denne sanntidsdataen til å justere vanntilførselen (åpne eller lukke ledeskovlene) i vannkraft, eller hvor fort rotoren går i vindkraft, for å opprettholde en stabil frekvens på strømmettet.

Diagrammet nedenfor viser hvordan en enkoder brukes i et vannkraftsystem for å måle turtall og gi tilbakemelding til kontrollsystemet.

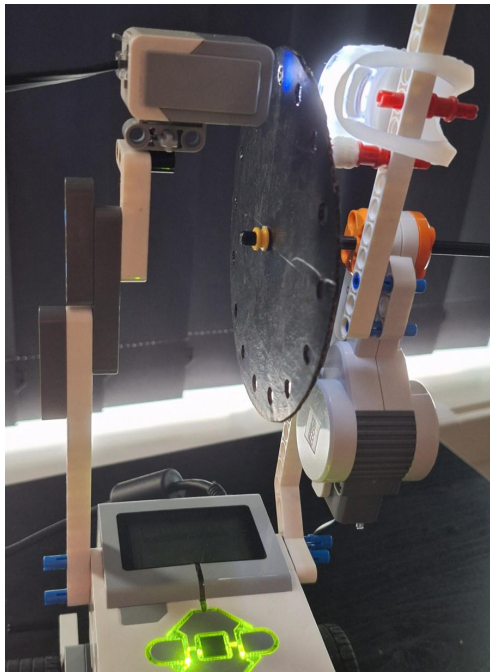


Figur 2: Enkoder i vannkraftverk

1.3 Oppsett

I dette prosjektet ble en optisk enkoder konstruert med LEGO, som vist i figur 3, for å måle rotasjonsbevegelsen til en LEGO EV3-motor. Motoren ble programmert i LEGO Mindstorms EV3 Software for å vise gjennomsnittlig rotasjonshastighet direkte i displayet.

Skiven ble festet til motorakselen for å måle aksens bevegelse. Et sykkellys ble benyttet som lyskilde for sensoren. For å oppnå et stabilt signal ble det innført en terskelverdi på 2 for lysnivået, og motorpådraget ble skalert med en faktor på 50. Valget av skalering og terskelverdi ble gjort empirisk, da denne kombinasjonen ga lavest støynivå i signalet. Forsøket ble utført i et mørkt rom for å redusere støy fra omgivelsene og sikre pålitelige målinger. Skiven er laget av tykk kartong med en radius på omtrent 6 cm og inneholder 16 jevnt fordelte hull med diameter 5 mm. Vinkelen mellom to påfølgende hull er 22.5° .



Figur 3: Egenkonstruert optisk enkoder

1.4 Metode

Vinkelhastigheten ω , som beskriver motorens rotasjonshastighet, beregnes som:

$$\omega = \frac{d\theta}{dt} \quad (1)$$

Vinkelakselerasjon α , som beskriver hvor raskt vinkelhastigheten endrer seg over tid, beregnet matematisk som

$$\alpha = \frac{d\omega}{dt} \quad (2)$$

Vinkelposisjonen S beregnes ved å telle antall registrerte pulser (N):

$$S = N \cdot \theta \quad (3)$$

MATLAB-koden for dette projektet inneholder tre deler: kantedeteksjon, numerisk beregning og filtering. Før målingen starter initialiseres variabler for vinkelposisjon, hastighet og akselerasjon, og startverdier settes til null. Kantedeteksjon vist i Kode 1 er benyttet for å registrere stigende flanker i signalet.

Listing 1: Kantedeteksjon

```
terskelverdi = 2;
signal(k) =double(Lys(k) > terskelverdi);

kant(k) = 0;
if k > 1
    % konstant verdi til neste oppdatering
    omega(k) = omega(k-1);
    alfa(k) = alfa(k-1);
    vinkel(k) = vinkel(k-1);
    if signal(k)==1 && signal(k-1)==0 %finn stigende
        flanke
        kant(k) = 1;
    end
end
end
```

I Kode 2 ble ω og α beregnet numerisk basert på de matematiske uttrykkene i henholdsvis ligning 1 og ligning 2, ved bruk av bakoverderivasjon. S ble beregnet numerisk ved bruk av Euler-integrasjon basert på det matematiske uttrykket i ligning 3.

Listing 2: Beregning av ingtegrasjon og bakoverderivasjon

```
if kant(k) == 1
    dt = t(k) - tp;
    vinkel(k) = vinkel(k-1) + theta; % Eulers-
        integrasjon
    omega(k) = theta / dt;% hastighet
    alfa(k) = (omega(k) - omega_last) / dt;%
        acceleration
    tp = t(k); % oppdatering
    omega_last = omega(k); % oppdatering data
end
```

For å redusere høyfrekvent støy ble et lavpassfilter med filterparameter $\beta = 0.2$ benyttet, som vist i Kode 3. Filterparameteren ble valgt empirisk for å oppnå en god balanse mellom støyreduksjon og responstid.

Listing 3: Filtre hastighet og akselerasjon

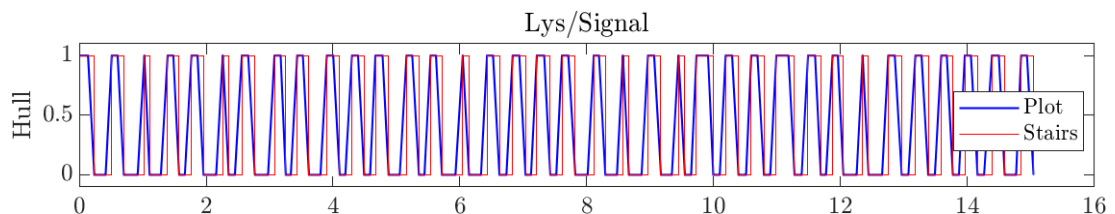
```

if k == 1
    omega_filt(k) = omega(k);
    alfa_filt(k) = alfa(k);
else
    omega_filt(k) = beta * omega(k) + ...
        (1 - beta) * omega_filt(k-1); % lavpassfilter
    alfa_filt(k) = beta * alfa(k) + ...
        (1 - beta) * alfa_filt(k-1); % lavpassfilter
end

```

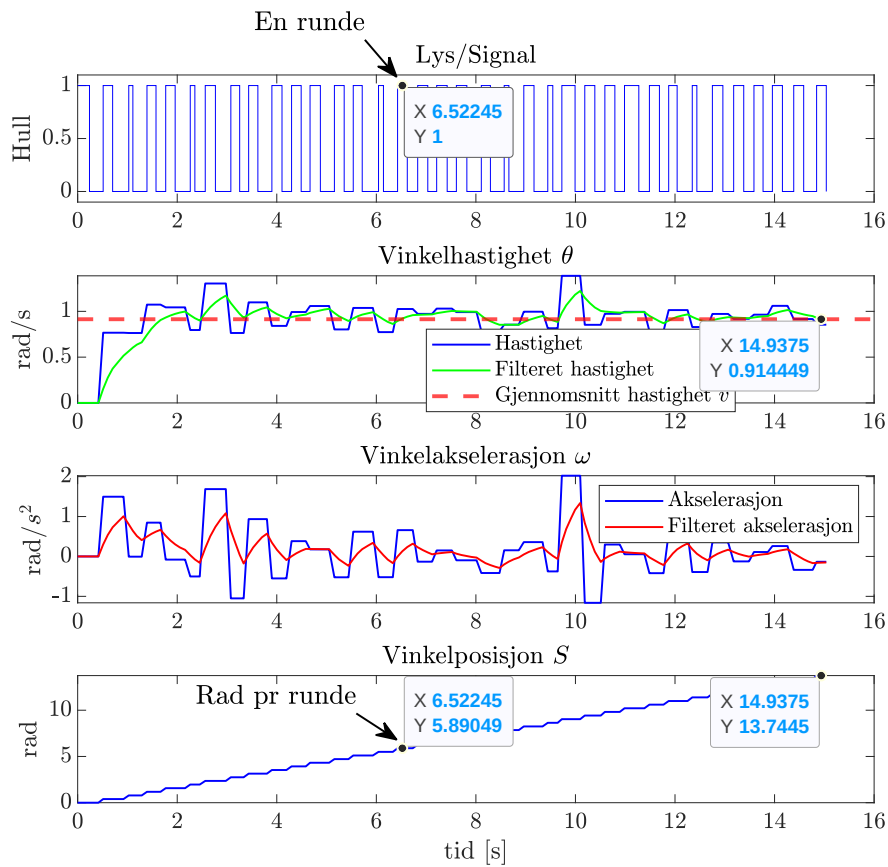
1.5 Resultat

Når signalet ble fremstilt med plot-funksjonen, fremstod det som et trapesformet signal på grunn av lineær sammenkobling mellom datapunktene. For å gi en mer korrekt visuell representasjon av det diskrete enkodersignalet ble stairs-funksjonen benyttet. Forskjellen mellom de to fremstillingsmåtene er vist i figur 4.



Figur 4: Trapes vs rektangel signal

Simuleringsresultatet fra den optiske enkoderen er vist i figur 5. Figuren viser rå-signalet samt beregnede bevegelsesstørrelser.



Figur 5: Simulering av signal, vinkelhastighet, akselerasjon og posisjon

Analyse og tolkning av simuleringen:

- Rå-signalet i øvre graf viser noe ujevnhet. Dette skyldes hovedsakelig den pulsbaserte og diskrete målemetoden, samt målestøy i sensoren.
- Små variasjoner i pulstid mellom påfølgende pulser forsterkes i beregningen av både vinkelhastighet og vinkelakselerasjon, siden disse størrelsene beregnes ved numerisk derivasjon. Et førsteordens IIR-lavpassfilter reduserer denne målestøyen ved å glatte ut de raske variasjonene.
- EV3-motoren har en gjennomsnittlig rotasjonshastighet på omtrent 0.9rad/s som er vist med den røde linjen i midtre graf. Denne verdien representerer motorens gjennomsnittlige bevegelse over måleintervallet.

1.6 Kryssverifisering:

Rad per runde

Avlesning fra den øvre grafen i figur 5 viser at 16 hull (høy-1) ble registrert innen 6.5s. Dette tilsvarer én full rotasjon av skiven, altså en vinkel på 2π radianer. For å kontrollere resultatet ser vi på den nedre grafen ved samme tidspunkt $t = 6,5s$, der rotasjonen er omtrent $5.89rad$. Dette er noe lavere enn den teoretiske verdien 2π . Avviket blir dermed:

$$\Delta S\% = \frac{2\pi - 5.89rad}{2\pi} \cdot 100\% \approx 6.3\%$$

Vinkelposisjon

Den totale vinkelposisjonen S etter omtrent 14.9s kan estimeres ved bruk av gjennomsnittshastigheten v som:

$$S = v \cdot t = 0.91rad/s \cdot 14.9s = 13.6rad$$

Dette stemmer godt overens med observasjon i den nederste grafen som viser $13.7rad$.

Motorfart i display

På display, vist i figur 6, viser den gjennomsnittlige farten på $-8.1rpm$ ¹.



Figur 6: Rotasjonshastighet i display

¹Negativt fortegn skyldes ulik definisjon av positiv retning i systemene.

Den simulerte gjennomsnittshastigheten vist i figur 5, målt i rad/s , konverteres til rpm :²:

$$v = 0.91 rad/s \cdot \frac{30}{\pi} \approx 8.7 rpm$$

Dette tilsvarer et avvik på

$$\Delta v\% = \frac{8.7 rpm - 8.1 rpm}{8.7 rpm} \cdot 100\% \approx 6.9\%$$

1.7 Diskusjon og konklusjon:

Råsignalet fremstår som trapesformet i stedet for ideelt firkantet, noe som skyldes begrenset sensorrespons, lysforhold og gradvise overganger når skiven roterer. Dette innebærer at overgangen mellom logiske nivåer ikke skjer momentant. Støy i signalet skyldes hovedsakelig variasjoner i lysintensitet, ufullstendig skjerming av sensoren og elektriske forstyrrelser fra omgivelsene.

Til tross for disse begrensningene viser systemet god overensstemmelse mellom teori og simulering. Metoden gir en forholdsvis presis beskrivelse av motorens utgangsbewegelse, men nøyaktigheten kunne vært forbedret gjennom bedre skjerming, høyere oppløsning og mer avansert filtrering.

²rmp står for Revolutions Per Minute. Konverteringsraten er: $1 rad/s = \frac{60}{2\pi} rpm = \frac{30}{\pi} rpm$

2 Vannlekkasjedetektor

Lodding, kretsanalyse og utvikling av en teknisk løsning basert på en reell problemstilling. Det ble gjennomført med utgangspunkt i en YouTube-video[3].

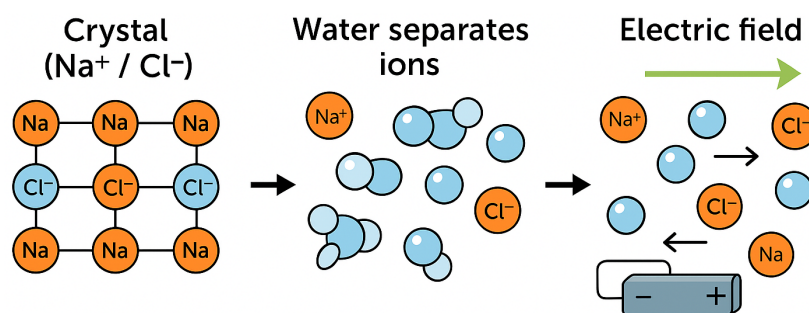
2.1 Problemstilling

Dette prosjektet gikk ut på å konvertere en gammel optisk røykvarsler til en enkel vannlekkasjedetektor. Ideen oppstod etter at det ble oppdaget en vannlekkasje under kjøkkenvasken hjemme. Målet var å undersøke om den eksisterende røykvarsler en kunne gjenbrukes til å varsle vannlekkasje.

2.2 Teori

Vannledningsevne

Springvann inneholder mineraler og salter som består av ioner bundet i et krystallgitter³. Vannmolekyler er polare og kan løse opp disse stoffene ved å frigjøre ionene. Når vannet kobles til en spenningskilde, oppstår et elektrisk felt som får de frie ionene til å bevege seg. Elektrisk strøm i vann er den samlede bevegelsen av ioner som drives av det elektriske feltet fra en spenningskilde. Prosessen er illustreret i figur7.



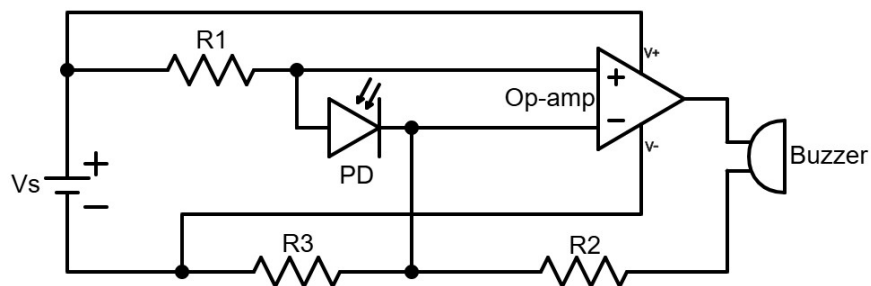
Figur 7: Hvordan ioner i vann leder elektrisk strøm. Blå bobler representerer vannmolekyler (KI-generert med brukerjusteringer)

Røykvarsler mekanism

³Et ion er et atom som har mistet eller fått elektroner, og dermed er en ladningsbærere. Positivt ladde ioner kalles kationer, og negativt ladde ioner kalles anioner.

En optisk røykvarsler bruker en infrarød LED og en fotodiode til å oppdage røyk. Når røykpartikler kommer inn i varsleren, spres lyset fra LED-en og treffer fotodioden. Dette lager et elektrisk signal som registreres av alarmkretsen og forsterkes av en operasjonsforsterker (op-amp). Deretter aktiveres en buzzer⁴, som lager alarmlyden.

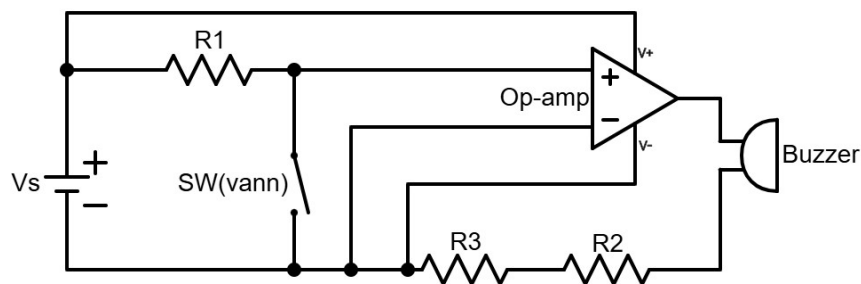
Den forenklet kretsen for røykvarsler en er vist i figur 8.



Figur 8: Kretsen for røykvarsler. Fotodioden (PD) fungerer som lyssensor i alarmen

Etter konverteringen erstattes den optiske sensoren med to prober som fungerer som en bryter. Når vann kommer i kontakt med begge probene, lukkes kretsen og det oppstår strøm. Dette trekker signalspenningen ned mot jord.

Den forenklede kretsen for den konverterte vanndetektoren er vist i figur 9.



Figur 9: Kretsen for vanndetektor. Vann fungerer som en bryter som lukker kretsen ved kontakt

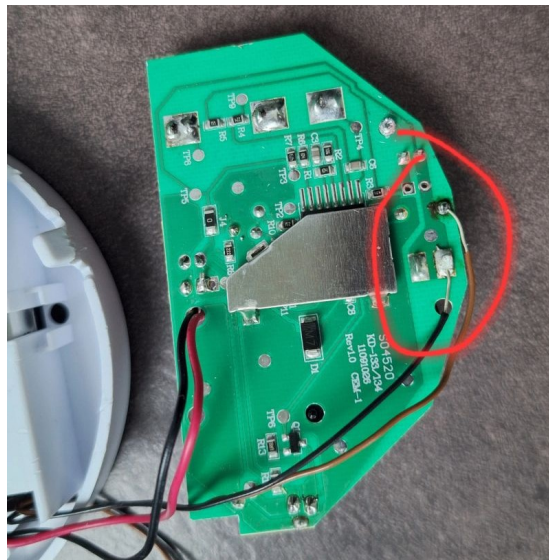
⁴En buzzer omdanner elektriske signaler til lyd

2.3 Eksperiment

Følgende utstyr ble benyttet:

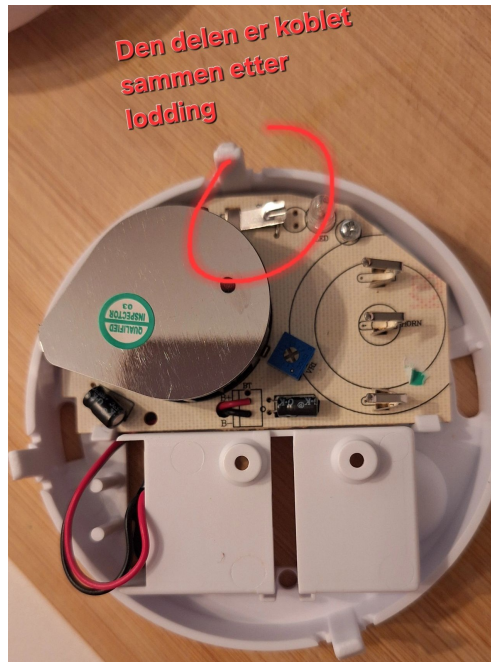
- Gammel røykvarsler med batteri
- Ledninger
- Loddebolt og loddetinn
- Skrutrekker

Røykvarsler en ble først demontert for å få tilgang til kretskortet. Deretter ble sensorinngangen identifisert, og to ledninger ble loddet til kretsen, som vist i figur 10. Disse ledningene fungerer som en bryter og plasseres i området som skal overvåkes.



Figur 10: Lodding av ekstern ledninger

Fysisk delen som ble kortsluttet av ledningene vises i bildet under:



Figur 11: Kortslutning av røyksensor

Den konverterte vanddetektor ble ferdigstilt og er vist i bildet under:



Figur 12: Kortslutning av røyksensor

Funksjonen ble testet ved å plassere elektrodene i vann. Alarmen ble aktivert umiddelbart når det oppstod elektrisk kontakt mellom ledningene.

2.4 Konklusjon

Prosjektet demonstrerer hvordan eksisterende elektronikk kan gjenbrukes til nye formål med enkle modifikasjoner. Videre arbeid kan inkludere forbedring av følsomheten, bruk av mer robuste materialer for elektrodene, samt implementering av trådløs varsling. Det er også mulig å utvikle systemet videre slik at det kan registrere både røyk og vann samtidig.

3 Strømkalkulator

Programmet ble utviklet i Python med et grafisk brukergrensesnitt (GUI) for å beregne energiforbruk.

3.1 Problemstilling

Hvordan kan man vurdere om det er kostbart å la elektriske enheter stå på over lengre tid? Gir det faktisk betydelige besparelser å slå av apparater som ikke er i bruk? Hvordan kan en enkel applikasjon gjøre slike beregninger forståelige og tilgjengelige for brukeren?

3.2 Utviklingsprosess

Dette prosjektet ble utviklet ved hjelp av kunstig intelligens. Funksjonalitet og brukerbehov ble beskrevet gjennom prompts, og KI ble brukt til å generere forslag til både kode og programstruktur.

Videre arbeid besto i å veilede, evaluere og forbedre løsningene som ble generert. Dette inkluderte testing, feilsøking og tilpasning av både programlogikk og brukergrensesnitt. Gjennom denne iterative prosessen ble prosjektet gradvis utviklet til en fungerende applikasjon som oppfyller kravene til beregning av strømkostnader.

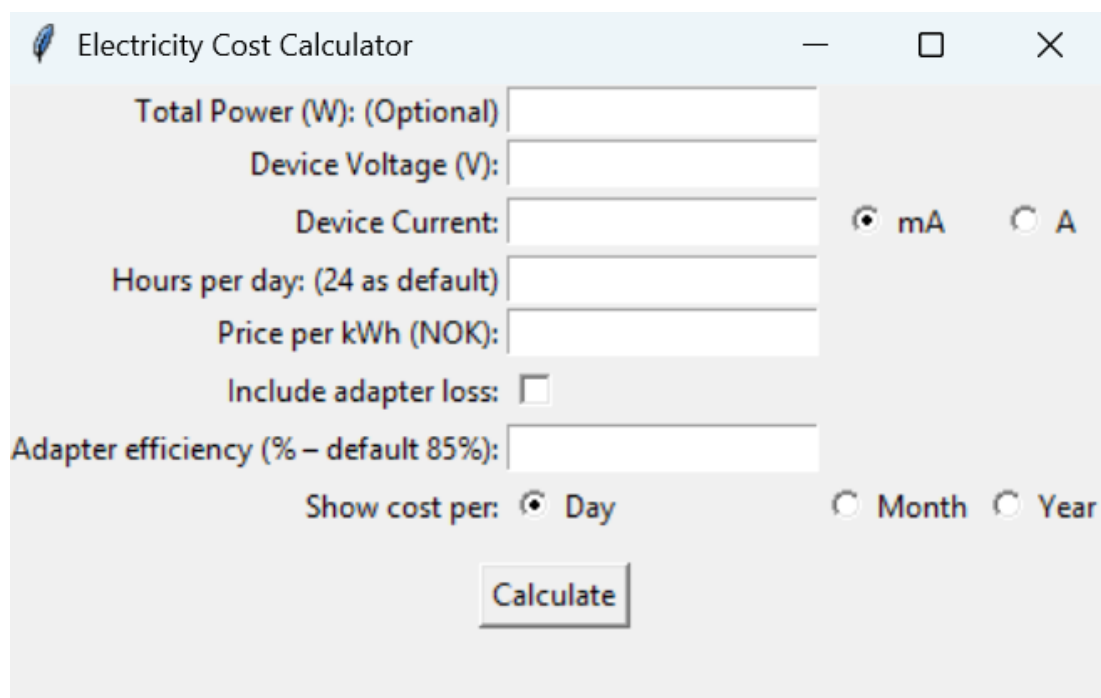
Prosjektet ble utviklet med mål om å kombinere praktisk nytteverdi med læring innen Python-programmering og effektberegning. Det ble lagt særlig vekt på å utforme et intuitivt grensesnitt som gjør det enkelt for brukeren å legge inn data og forstå resultatene.

En av hovedutfordringene var å håndtere ulike metoder for beregning av effektforbruk uten å skape forvirring for brukeren. Dette ble løst ved å la brukeren velge mellom å oppgi total effekt eller spenning og strømstyrke, samtidig som irrelevante inndatafelt automatisk deaktiveres. Det ble også lagt vekt på nøyaktige beregninger som tar hensyn til adaptereffektivitet og ulike tidsperioder.

3.3 Funksjon

- Angi enten total effekt (W) eller enhetens spenning og strømstyrke
- Inkludere adaptereffektivitet for å ta hensyn til energitap
- Beregne strømkostnad per dag, måned eller år
- Tilpassbar strømpris (NOK eller annen valuta)
- Brukervennlig grensesnitt for raske beregninger

Skjermbildet vist i figur 13 viser applikasjonens brukergrensesnitt (GUI), utviklet med Tkinter. Brukeren legger inn nødvendig informasjon i de ulike inndatafeltene, som effekt (W) eller spenning (V) og strømstyrke (A). Denne informasjonen kan vanligvis finnes på enhetens adapter, strømforsyning eller produktets merkeskilt.



The screenshot shows a Tkinter window titled "Electricity Cost Calculator". It contains the following elements:

- Input fields:**
 - Total Power (W): (Optional)
 - Device Voltage (V):
 - Device Current:
 - Hours per day: (24 as default)
 - Price per kWh (NOK):
 - Adapter efficiency (% - default 85%):
- Radio buttons:**
 - Next to "Device Current": ☒ mA and ☐ A
 - Next to "Show cost per:": ☒ Day, ☐ Month, and ☐ Year
- Buttons:**
 - A "Calculate" button at the bottom.

Figur 13: Brukergrensesnitt for strømkalkulatoren

Denne applikasjonen er nyttig for huseiere, hobbybrukere og andre som ønsker å få bedre oversikt over og redusere energikostnader.

Programmet kan lastes ned fra mitt GitHub-repositorium:
<https://github.com/Circuit-Commander/Electricity-Cost-Calculator>

4 Analyse av strømforbruk i Excel og Access

4.1 Formål

Før det kan gjennomføres analyser av lastprofilering og kapasitetsplanlegging i strømmettet, må rådata om strømforbruk kontrolleres, struktureres og analyseres. Dette projektet omhandler dette forarbeidet.

Det ble benyttet strømforbruksdata fra Elhub, “Forbruk per prisområde, gruppe og time”[4]. Datasettet inneholder timesoppløst strømforbruk målt i kWh, med angitt starttid og sluttid for hver måling. Dataene er fordelt på fem prisområder (NO1–NO5) og fem forbruksgrupper: primær industri, sekundær industri, tertiære tjenester, husholdning samt hytter og fritidseiendommer. På grunn av datamengden viser kun et utdrag av tabellen, vist i Figur 14.

START_TIME	END_TIME	PRICE_AREA	CONSUMPTION_GROUP	VOLUME_KWH	METERINGPOINT_COUNT
01/01/2021 00:00	01/01/2021 01:00	NO1	Cabins and holiday properties	177071.569	100607
01/01/2021 00:00	01/01/2021 01:00	NO1	Household	2366888.785	1050893
01/01/2021 00:00	01/01/2021 01:00	NO1	Primary Industry	70166.621	10332
01/01/2021 00:00	01/01/2021 01:00	NO1	Secondary Industry	693011.732	20866
01/01/2021 00:00	01/01/2021 01:00	NO1	Tertiary Service	1123886.685	97386
01/01/2021 00:00	01/01/2021 01:00	NO2	Cabins and holiday properties	142641.701	102095
01/01/2021 00:00	01/01/2021 01:00	NO2	Household	1425141.028	591610

Figur 14: Utdrag av strømdatatabellen

NO1–NO5 representerer henholdsvis Østlandet, Sørlandet, Midt-Norge, Nord-Norge og Vestlandet. Figur 15 illustrerer den geografiske fordelingen.

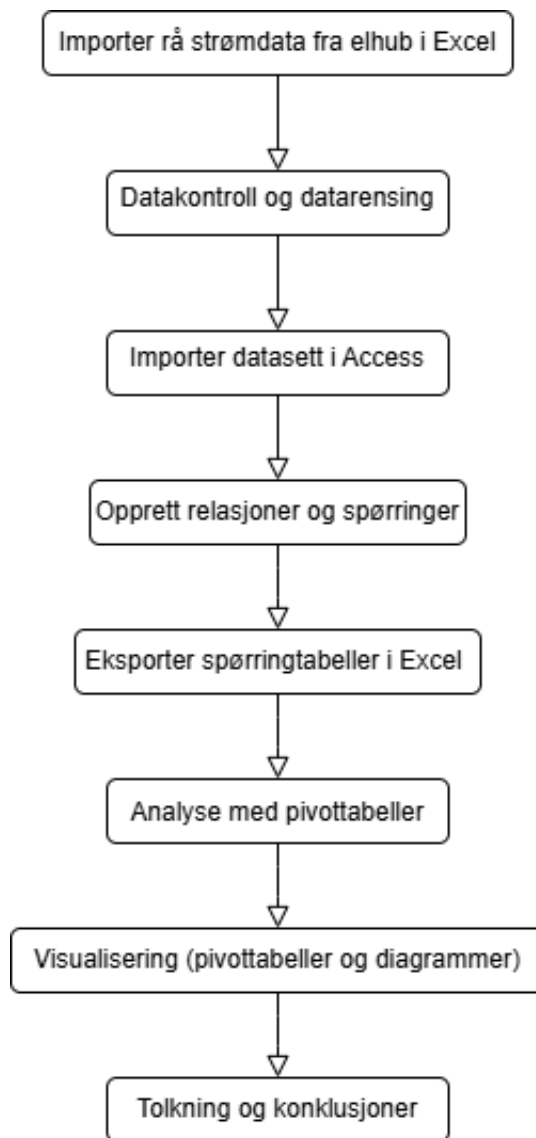


Figur 15: Geografisk fordeling av prisområdene[5]

Ved hjelp av Microsoft Excel og Access ble følgende problemstillinger undersøkt:

- Hvilken forbruksgruppe som har høyest strømforbruk.
- Hvilket prisområde som har størst strømforbruk.
- Når på døgnet strømforbruket er høyest.
- Hvordan strømforbruket fordeler seg mellom forbruksgrupper og prisområder.

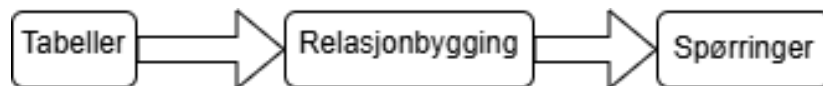
Den metodiske arbeidsprosessen er illustrert i flytskjemaet nedenfor.



Figur 16: Flytskjema som viser arbeidsprosessen

4.2 Databehandling i Access

Som vist i Figur 16 ble datasettet først kontrollert i Excel. Det ble ikke funnet duplikater eller manglende verdier i datasettet. Etter kvalitetssikringen ble dataene importert til Access, hvor de ble strukturert i en relasjonsdatabase. Videre ble relevant informasjon hentet ut ved hjelp av spørringer opprettet i Query Design. Prosessen er vist i Figur 17:



Figur 17: Arbeidsprosess i Access

Tabeller

Etter import ble hovedtabellen opprettet som *tblConsumption*. I tillegg ble egne tabeller opprettet for prisområder og forbruksgrupper. Databasen består dermed av følgende tabeller:

- *tblConsumption*
- *tblPrice*
- *tblGroup*

Tabellene *tblPrice* og *tblGroup* er vist henholdsvis i Figur 1 og Figur 2. *tblConsumption* inneholder det komplette datasettet.

PriceAreaID ▾	PriceArea ▾
1	NO1
2	NO2
3	NO3
4	NO4
5	NO5

Tabell 1: Prisområder

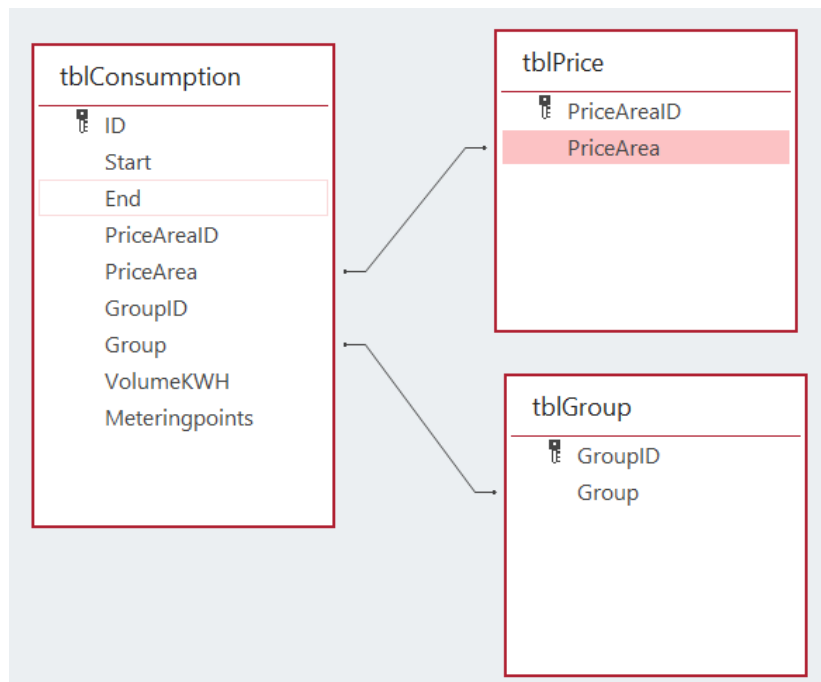
GroupID ▾	Group ▾
1	Primary Industry
2	Secondary Industry
3	Tertiary Service
4	Household
5	Cabins and holiday properties

Tabell 2: Forbruksgrupper

For å etablere relasjoner mellom tabellene ble fremmednøkler (*PriceAreaID* og *GroupID*) lagt til i *tblConsumption*. Disse feltene ble koblet mot primærnøklerne i *tblPrice* og *tblGroup* for å sikre korrekt datakobling og referensiell integritet.

Relasjoner

Hver tabell ble tildelt en primærnøkkel som identifiserer hver post unikt. Relasjonene mellom tabellene ble etablert ved bruk av primær- og fremmednøkler, som vist i Figur 18.



Figur 18: Relasjon mellom tabellene

Spørringer

Etter at relasjonene mellom tabellene var etablert, ble det opprettet fire spørringer som deretter ble eksportert til Excel, hvor dataene ble analysert videre ved hjelp av pivottabeller og visualisering. Spørringene for totalt forbruk per prisområde og per forbrukgruppe er vist henholdsvis i tabell 3 og tabell 4. De øvrige, mer omfattende spørreresultatene er inkludert som vedlegg.

PriceAreaID ▾	PriceArea ▾	SumOfVolumeKWH ▾
1	NO1	154199590656.115
2	NO2	167161607451.639
3	NO3	129566725284.256
4	NO4	88889935515.8279
5	NO5	76954929554.4842

Tabell 3: Totalt strømforbruk per prisområdet

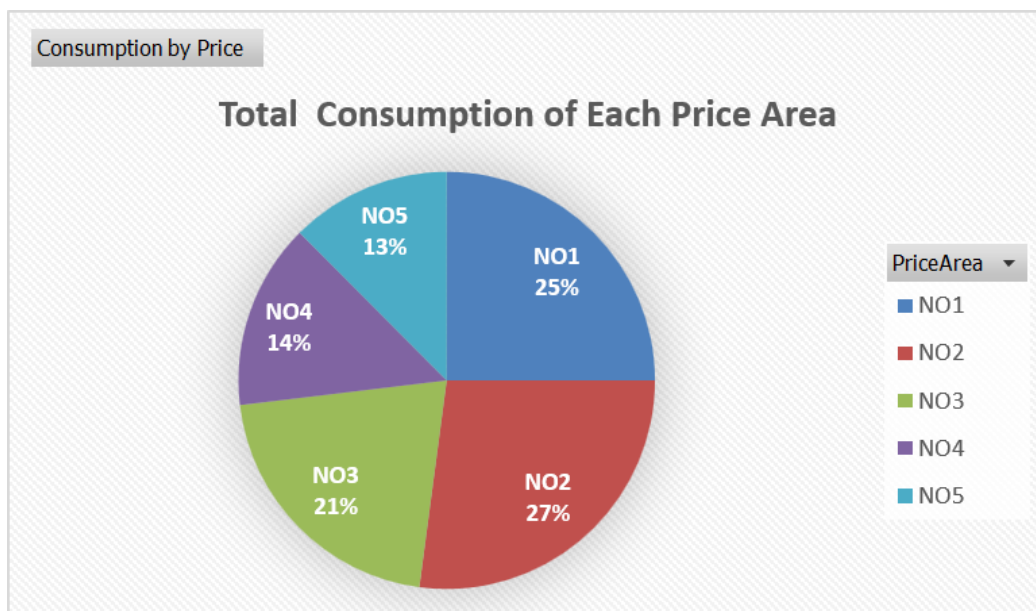
GroupID	Group	SumOfVolumeKWH
1	Primary Industry	11621838873.114
2	Secondary Industry	287038983352.228
3	Tertiary Service	126122505646.014
4	Household	180180432850.064
5	Cabins and holiday properties	11809027740.9091

Tabell 4: Totalt strømforbruk per forbruksgroup

4.3 Analyse og visualisering i Excel

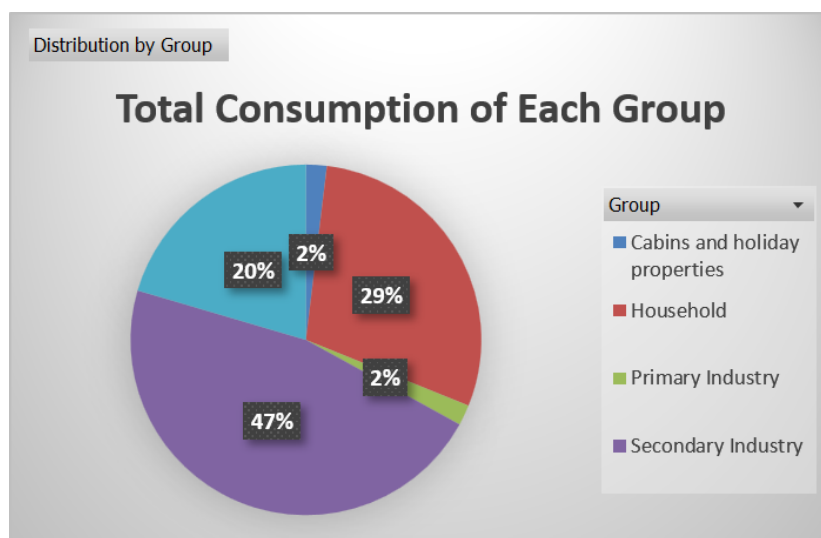
Spørreresultatene fra Access ble eksportert til Microsoft Excel for videre analyse. Det ble opprettet pivottabeller og diagrammer for å sammenstille og visualisere strømforbruket.

Figur 19 viser det totale strømforbruket fordelt på prisområder. Strømforbruket er relativt likt i NO1, NO2 og NO3, men NO2 har det høyeste totale forbruket. NO4 og NO5 har et lavere samlet forbruk. Dette kan indikere regionale forskjeller i energibehov, som kan være påvirket av ulik befolkningstetthet, næringsstruktur og industrivirksomhet. Det tilgjengelige datasettet gir imidlertid ikke grunnlag for å fastslå hvilke faktorer som faktisk forklarer forskjellene.



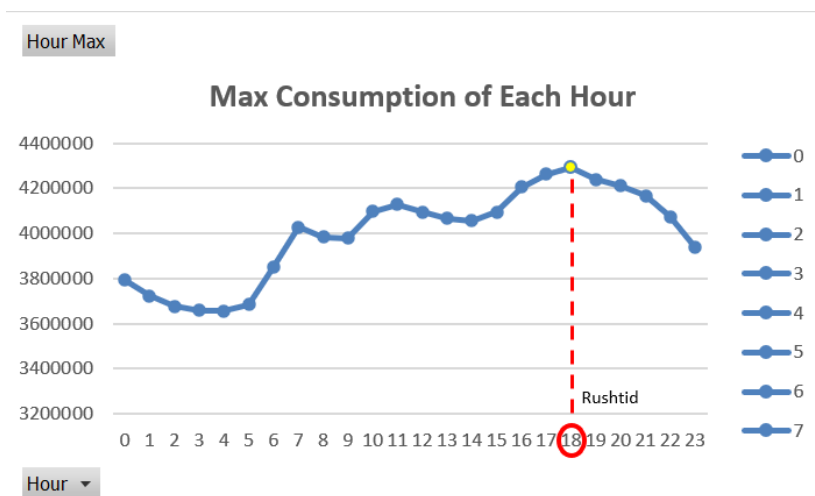
Figur 19: Totalt strømforbruk fordelt på prisområder

Figur 20 viser strømforbruket fordelt på ulike forbruksgrupper. Sekundærnæringen har det høyeste strømforbruket og utgjør litt under halvparten av det totale forbruket, mens husholdningene står for omtrent en tredjedel. Det høye strømforbruket i sekundærnæringen kan forklares med at industrivirksomheter ofte har et kontinuerlig energibehov knyttet til produksjonsutstyr og prosessanlegg. Husholdningene representerer også en betydelig andel av forbruket på grunn av det store antallet brukere samt et vedvarende behov for blant annet oppvarming, belysning og matlaging. Resultatene illustrerer den sentrale rollen industrien spiller i det samlede strømforbruket, samtidig som de viser at husholdningene utgjør en viktig del av den totale etterspørselen etter elektrisitet.



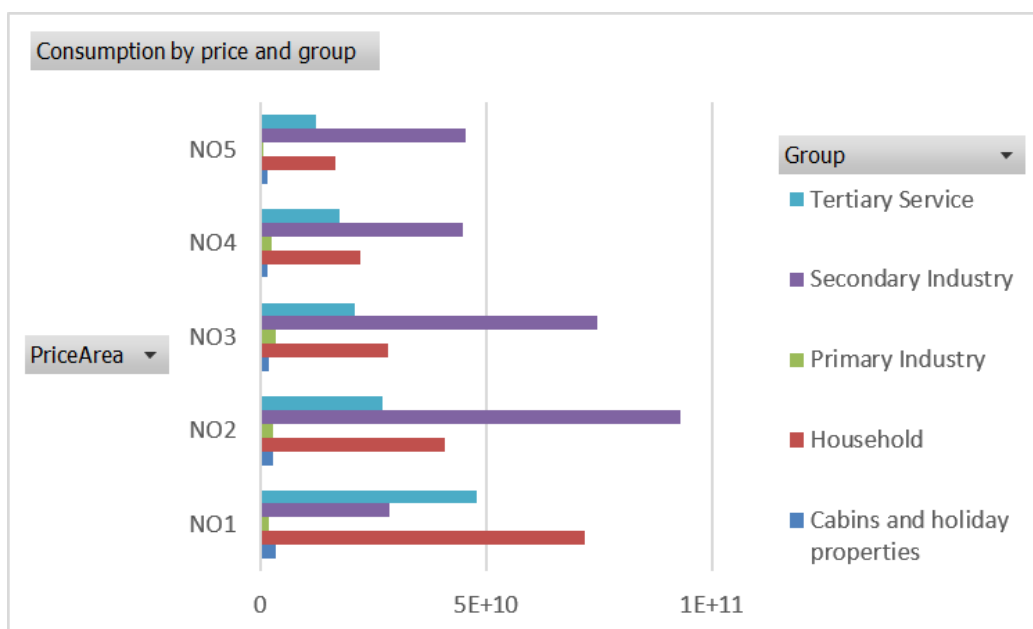
Figur 20: Totalt strømforbruk fordelt på forbruksgrupper

Figur 23 viser variasjonen i strømforbruk gjennom døgnet. Det høyeste forbruket oppstår rundt kl. 18. Dette samsvarer med tidspunktet da mange husholdninger benytter elektrisk oppvarming, matlaging og andre elektriske apparater samtidig. Slike belastningstopper er viktige ved dimensjonering og drift av distribusjonsnett, ettersom nettet må kunne håndtere perioder med høy belastning.



Figur 21: Strømforbruk fordelt på time i døgnet

Figur 22 viser strømforbruket fordelt på både forbruksgrupper og prisområder. Sekundær industri har det høyeste forbruket i prisområdene NO2–NO5, mens householdning har høyest forbruk i NO1. Dette viser at ulike regioner kan ha forskjellige forbruksmønstre, avhengig av næringsstruktur, befolkningstetthet og lokal aktivitet.



Figur 22: Strømforbruk fordelt på forbruksgrupper og prisområder

4.4 Konklusjon

I dette prosjektet ble strømdata fra Elhub analysert ved hjelp av Microsoft Access og Excel. Dataene ble strukturert gjennom tabeller, relasjoner og spørringer før resultatene ble visualisert med pivottabeller og diagrammer.

Analysen viste at prisområdet NO2 hadde det høyeste totale strømforbruket, sekundær industri var den største forbruksgruppen, og at forbrukstoppen oppstod rundt kl. 18. De identifiserte belastningstoppene kan brukes som grunnlag for videre vurdering av kapasitetsbehov og belastningsutvikling i strømmettet. For en fullstendig kapasitetsanalyse kreves imidlertid mer detaljerte data om faktisk effektbelastning og kapasitet i de enkelte nettkomponentene.

Resultatene illustrerer hvordan analyse av forbruksdata kan bidra til økt forståelse av belastningsmønstre i kraftsystemet og danne grunnlag for framtidig nettplanlegging og kapasitetsutvikling.

Refleksjon

Gjennom disse prosjektene har jeg utviklet en bedre teknisk forståelse innen elektronikk og en mer strukturert arbeidsmetodikk. Jeg har fått erfaring med å planlegge, gjennomføre og dokumentere prosjekter – fra idéutvikling til ferdig løsning.

For å presentere ideene på en visuell og oversiktlig måte har jeg benyttet ulike tegneprogrammer for kretsanalyse og flytskjema. Testing, feilsøking og kontinuerlig forbedring av løsningene har stått sentralt gjennom hele arbeidet.

Prosjektene har også styrket min evne til å se sammenhenger mellom data, teknologi og praktiske behov. Jeg har blitt bedre til å sette meg inn i nye verktøy, tenke kreativt og arbeide tverrfaglig for å finne gode løsninger på komplekse utfordringer.

Referanser

Referanser

- [1] Quantum Devices (n.d.). *Optical Encoder Basics: What Is an Optical Encoder?*
Tilgjengelig fra: <https://www.quantumdev.com/resource-library/optical-encoder-basics-what-is-an-optical-encoder/>
- [2] AKM (n.d.). *Rotation Angle Sensor: Type and Mechanism 2*.
Tilgjengelig fra: <https://www.akm.com/eu/en/products/rotation-angle-sensor/tutorial/type-mechanism-2/>
- [3] DDIY GUY, Danny Jose (n.d.).
Tilgjengelig fra: <https://www.youtube.com/watch?v=Umkw15FETno&list=PLSZonzWHNUBA4R06sT7ZUckkuLe-UR-Pz>
- [4] Elhub (n.d.). *Forbruk per prisområde, gruppe og time*. Datakatalog.
Tilgjengelig fra: <https://elhub.no/data-og-innsikt/datakatalog>
- [5] Lyse (n.d.). *Hvorfor har vi ulike prisområder?*
Tilgjengelig fra: <https://www.lyse.no/kundeservice/artikler/8944576174097-hvorfor-har-vi-ulike-prisomr%C3%A5der> ““

Vedlegg

A Rushtid

Hour ▼	MaxOfVolun ▼
0	3794063.457
1	3723049.491
2	3676648.783
3	3657881.611
4	3656543.407
5	3683672.628
6	3849687.604
7	4026986.934
8	3984381.515
9	3979901.788
10	4098764.378
11	4129979.803
12	4095850.976
13	4065706.643
14	4057049.024
15	4094274.491
16	4206810.407
17	4262178.49
18	4290733.863
19	4240444.169
20	4212432.159
21	4166282.65
22	4073737.034
23	3938377.74

Figur 23: Strømforbruk fordelt på tidspunkt

B Spørringsresultat for forbruk per prisområde og forbruksgruppe

Group ▼	PriceArea ▼	SumOfVolumeKWH ▼
Cabins and holiday properties	NO1	3616027938.731
Cabins and holiday properties	NO2	2944068365.17099
Cabins and holiday properties	NO3	1881791536.627
Cabins and holiday properties	NO4	1708279282.18799
Cabins and holiday properties	NO5	1658860618.19199
Household	NO1	71782625272.3553
Household	NO2	41013046945.5237
Household	NO3	28320145170.1969
Household	NO4	22328142309.843
Household	NO5	16736473152.1451
Primary Industry	NO1	2016150255.03501
Primary Industry	NO2	3004861682.93101
Primary Industry	NO3	3581746714.35701
Primary Industry	NO4	2419770528.90101
Primary Industry	NO5	599309691.889998
Secondary Industry	NO1	28751755806.33
Secondary Industry	NO2	93118223942.0486
Secondary Industry	NO3	74706499893.8417
Secondary Industry	NO4	44945794473.9239
Secondary Industry	NO5	45516709236.0802
Tertiary Service	NO1	48033031383.6654
Tertiary Service	NO2	27081406515.9679
Tertiary Service	NO3	21076541969.231
Tertiary Service	NO4	17487948920.972
Tertiary Service	NO5	12443576856.176

Figur 24: Forbruk per prisområde og gruppe

C Python og MATLAB koder

Listing 4: Enkoder

```

clear; close all      % Alltid lurt å rydde workspace opp først
online = false;       % Online mot EV3 eller mot lagrede data?
plotting = false;     % Skal det plottes mens forsøket kjøres
filename = 'encoder_korttid.mat'; % Data ved offline

if online
    mylego = legoev3('USB');
    joystick = vrjoystick(1);
    [JoyAxes, JoyButtons] = HentJoystickVerdier(joystick);

    % Hvilke sensorer er koplet til?
    myColorSensor = colorSensor(mylego);
    resetRotationAngle(myColorSensor);
    motorD = motor(mylego, 'D');
    motorD.resetRotation;
else
    % Dersom online=false lastes datafil.
    load(filename)
end

fig1=figure;
set(gcf, 'units', 'normalized', 'outerposition', [0.1 0.3 0.6
0.6])
drawnow

% setter skyteknapp til 0, og initialiserer tellevariabel k
JoyMainSwitch=0;
k=0;
%-----
tp_last = 0;
omega_last = 0;
omega = zeros(1, 10000); % Pre-alloker for å unngå treghet
alfa = zeros(1, 10000);
vinkel = zeros(1, 10000); % Pre-allocate for the plot

while ~JoyMainSwitch

    %                               GET TIME AND MEASUREMENT
    % Få tid og målinger fra sensorer, motorer og joystick

    k=k+1;                          % oppdater tellevariabel

    if online
        if k==1

```

```

        % Spiller av lyd slik at du vet at innsamlingen
        har startet
        playTone(mylego,500,0.1)    % 500Hz i 0.1 sekund
        tic                        % Starter stoppeklokke
        t(1) = 0;
    else
        t(k) = toc;    % Henter ut medgått tid
    end

    % Sensorer, bruk ikke Lys(k) og LysDirekte(k)
    samtidig
    Lys(k) = double(readLightIntensity(myColorSensor,'
        ambient'));
    %VinkelPosMotorD(k) = double(motorC.readRotation);

    % Data fra styrestikke. Utvid selv med andre knapper
    og akser.
    % Bruk filen joytest.m til å finne koden for
    knappene og aksene.
    [JoyAxes,JoyButtons] = HentJoystickVerdier(joystick)
    ;
    JoyMainSwitch = JoyButtons(1);
    JoyForover(k) = JoyAxes(2);
else
    if k==length(t)
        JoyMainSwitch=1;
    end

    if plotting
        pause(0.03)
    end
end
end

```

-

```

%                CONDITIONS, CALCULATIONS AND SET
Nhull= 16;
theta=2*pi/Nhull;
threshold = 2;

% ++++++signal+++++
signal(k) = Lys(k) > threshold;
square(k)=signal(k) > 0.5;
if k > 1
    omega(k) = omega(k-1);
    alfa(k) = alfa(k-1);

```

```

        vinkel(k) = vinkel(k-1); % Keep the angle constant
                                between pulses
    end
    %+++++TID+++++

    if k > 1 && signal(k-1)==0 && signal(k)==1
        % Calculate total angular distance (radians)
        vinkel(k) = vinkel(k-1) + theta;

        dt = t(k) - tp_last;
        omega(k) = theta / dt;% hastighet
        alfa(k) = (omega(k) - omega_last) / dt;%
                acceleration

        % Update for next time
        tp_last = t(k);
        omega_last = omega(k);

    end
    % +++++vinkelhastighet+++
    if online
        JoyForward = 50 * JoyAxes(2);
        motorD.Speed = JoyForward;
        start(motorD)
    end
    %-----
    if plotting || JoyMainSwitch
        figure(fig1)

        subplot(4,1,1)
        plot(t(1:k),signal(1:k));
        hold on
        stairs(t(1:k),square(1:k))
        hold off
        ylim([-0.1 1.1]); ylabel('Hull'); title('Lys/Signal'
        )

        subplot(4,1,2)
        plot(t(1:k),omega(1:k)); ylabel('rad/s'); title('
        Vinkelhastighet $\theta$')

        subplot(4,1,3)
        plot(t(1:k),alfa(1:k)); ylabel('rad/$s^2$'); title(
        'Vinkelakselasjon $\omega$')

```

```

        subplot(4,1,4)
        plot(t(1:k),vinkel(1:k)); ylabel('rad'); title('
            Vinkel $$$');
        xlabel('tid [s]')

        drawnow
    end
    %-----
end

%                STOP MOTORS
if online
    stop(motorD);
end

```

Listing 5: Elektrisitet kakulator

```

import tkinter as tk
from tkinter import messagebox

MAX_DEVICE_VOLTAGE = 12.0
DEFAULT EFFICIENCY = 0.85

def toggle_voltage_current(event=None):
    if power_entry.get().strip():
        voltage_entry.config(state="disabled")
        current_entry.config(state="disabled")
    else:
        voltage_entry.config(state="normal")
        current_entry.config(state="normal")

def calculate():
    voltage_entry.config(bg="white")

    # --- Read voltage early for warning ---
    voltage_value = None
    if voltage_entry.get():
        try:
            voltage_value = float(voltage_entry.get())
        except ValueError:
            messagebox.showerror("Input Error", "Voltage
                must be a number.")
        return

```

```

        if voltage_value > MAX_DEVICE_VOLTAGE:
            voltage_entry.config(bg="#ffcccc")
            proceed = messagebox.askyesno(
                "Possible Adapter Voltage",
                f"You entered {voltage_value} V.\n\n"
                "For small devices, the device voltage is"
                "usually 1â€“12 V.\n"
                "This may be the adapter voltage.\n\n"
                "Do you want to continue using this value?"
            )
            if not proceed:
                return

# --- Required fields ---
try:
    # Check if the entry is empty; if so, default to 24
    hours_input = hours_entry.get().strip()
    hours = float(hours_input) if hours_input else 24.0
    price_per_kwh = float(price_entry.get())
except ValueError:
    messagebox.showerror("Input Error", "Please enter
        valid numbers for hours and price.")
    return

period = period_var.get()

# --- Power calculation ---
power_w = None

if power_entry.get().strip():
    try:
        power_w = float(power_entry.get())
    except ValueError:
        messagebox.showerror("Input Error", "Total power
            must be a number.")
        return
else:
    if not (voltage_entry.get() and current_entry.get()):
        :
        messagebox.showerror(
            "Input Error",
            "Enter either Total Power (W)\n"
            "or Device Voltage and Current."
        )
        return
    try:

```

```

        current = float(current_entry.get())
        current_a = current / 1000 if current_unit.get()
            == "mA" else current
        power_w = voltage_value * current_a
    except ValueError:
        messagebox.showerror("Input Error", "Current
            must be a number.")
        return

# --- Adapter efficiency ---
try:
    efficiency = (
        float(efficiency_entry.get()) / 100
        if efficiency_entry.get().strip()
        else DEFAULT_EFFICIENCY
    )
except ValueError:
    messagebox.showerror("Input Error", "Efficiency must
        be a number.")
    return

if not (0 < efficiency <= 1):
    messagebox.showerror("Input Error", "Efficiency must
        be between 0 and 100%.")
    return

include_adapter = adapter_var.get()

if include_adapter:
    total_power = power_w / efficiency
    loss_power = total_power - power_w
else:
    total_power = power_w
    loss_power = 0

# --- Cost calculation ---
energy_day = total_power * hours / 1000
cost_day = energy_day * price_per_kwh
cost_month = cost_day * 30
cost_year = cost_day * 365

loss_cost_day = loss_power * hours / 1000 *
    price_per_kwh
loss_cost_month = loss_cost_day * 30
loss_cost_year = loss_cost_day * 365

```

```

# --- Output ---
if period == "day":
    result = f"Electricity cost per day: {cost_day:.2f}
    NOK"
    if include_adapter:
        result += f"\nAdapter loss per day: {
            loss_cost_day:.2f} NOK"
elif period == "month":
    result = f"Electricity cost per month: {cost_month
        :.2f} NOK"
    if include_adapter:
        result += f"\nAdapter loss per month: {
            loss_cost_month:.2f} NOK"
else:
    result = f"Electricity cost per year: {cost_year:.2f
        } NOK"
    if include_adapter:
        result += f"\nAdapter loss per year: {
            loss_cost_year:.2f} NOK"

result_text.set(result)

# ----- GUI -----
root = tk.Tk()
root.title("Electricity Cost Calculator")

tk.Label(root, text="Total Power (W): (Optional)").grid(row
    =0, column=0, sticky="e")
power_entry = tk.Entry(root)
power_entry.grid(row=0, column=1)
power_entry.bind("<KeyRelease>", toggle_voltage_current)

tk.Label(root, text="Device Voltage (V):").grid(row=1,
    column=0, sticky="e")
voltage_entry = tk.Entry(root)
voltage_entry.grid(row=1, column=1)

tk.Label(root, text="Device Current:").grid(row=2, column=0,
    sticky="e")
current_entry = tk.Entry(root)
current_entry.grid(row=2, column=1)

current_unit = tk.StringVar(value="mA")
tk.Radiobutton(root, text="mA", variable=current_unit, value
    ="mA").grid(row=2, column=2)

```

```

tk.Radiobutton(root, text="A", variable=current_unit, value
    ="A").grid(row=2, column=3)

tk.Label(root, text="Hours per day: (24 as default)").grid(
    row=3, column=0, sticky="e")
hours_entry = tk.Entry(root)
hours_entry.grid(row=3, column=1)

tk.Label(root, text="Price per kWh (NOK):").grid(row=4,
    column=0, sticky="e")
price_entry = tk.Entry(root)
price_entry.grid(row=4, column=1)

tk.Label(root, text="Include adapter loss:").grid(row=5,
    column=0, sticky="e")
adapter_var = tk.BooleanVar()
tk.Checkbutton(root, variable=adapter_var).grid(row=5,
    column=1, sticky="w")

tk.Label(root, text="Adapter efficiency (% â€“ default 85%)
    :").grid(row=6, column=0, sticky="e")
efficiency_entry = tk.Entry(root)
efficiency_entry.grid(row=6, column=1)

tk.Label(root, text="Show cost per:").grid(row=7, column=0,
    sticky="e")
period_var = tk.StringVar(value="day")
tk.Radiobutton(root, text="Day", variable=period_var, value
    ="day").grid(row=7, column=1, sticky="w")
tk.Radiobutton(root, text="Month", variable=period_var,
    value="month").grid(row=7, column=2, sticky="w")
tk.Radiobutton(root, text="Year", variable=period_var, value
    ="year").grid(row=7, column=3, sticky="w")

tk.Button(root, text="Calculate", command=calculate).grid(
    row=8, column=0, columnspan=4, pady=10
)

result_text = tk.StringVar()
tk.Label(root, textvariable=result_text, font=("Arial", 12,
    "bold"), justify="left").grid(
    row=9, column=0, columnspan=4
)

toggle_voltage_current()
root.mainloop()

```